

PayrollFiti

Implementation — Stage 1 Complete Database Schema and Migrations

Laravel 13 + PostgreSQL + Redis backend, real runnable code
Migrations, Models, Factories, Seeders — tenancy, identity, payroll core, leave, audit, idempotency

September 2026

Table of Contents

PayrollFiti — Implementation

STAGE 1 — Complete Database Schema and Migrations

1. Entity Inventory (reconciled against the source guide + Part A)
 2. Dependency Order (Stage 1 migrations)
 3. Migrations
 4. Enums
 5. Tenancy Foundation (used by every model below)
 6. Identity
 7. Employees
 8. Payroll (compensation configuration + run/entry models)
 9. Leave
 10. Audit
 11. Idempotency
 12. Factories
 13. Seeders
 14. Stage 1 Completeness Check
- Stage 1 Persistent Implementation Checklist

PayrollFiti — Implementation

STAGE 1 — Complete Database Schema and Migrations

1. Entity Inventory (reconciled against the source guide + Part A)

Every table below is either **[SOURCE]** (a Prisma model in the original guide), **[LARAVEL]** (how Laravel expresses a source requirement — e.g. Sanctum's token table instead of a hand-rolled `ApiKey` model), or **[NEW]** (an additive table with a stated safe default, per Part A). This stage implements the tables marked  **Stage 1** below; everything marked **» Stage 1 continuation** is listed in the checklist at the end and picked up in the next response, per the file-boundary protocol.

#	Table	Classification	Status
1	tenants	[SOURCE]	 Stage 1
2	companies	[SOURCE]	 Stage 1
3	users	[SOURCE]	 Stage 1
4	roles , permissions , model_has_roles , model_has_permissions , role_has_permissions	[LARAVEL] (spatie/laravel- permission, tenant-scoped via its native "teams" feature)	 Stage 1
5	personal_access_tokens	[LARAVEL] (Sanctum — replaces the source's separate <code>ApiKey</code> model, §18.1 of the architecture doc)	 Stage 1
6	employees	[SOURCE]	 Stage 1
7	contracts	[SOURCE]	 Stage 1
8	salary_structures	[SOURCE]	 Stage 1
9	salary_components	[SOURCE]	 Stage 1
10	salary_structure_compon ents	[SOURCE]	 Stage 1
11	onboarding_tasks	[SOURCE]	 Stage 1
12	payroll_periods	[NEW — confirm, safe default applied]	 Stage 1
13	payroll_runs	[SOURCE]	 Stage 1
14	payroll_entries	[SOURCE]	 Stage 1
15	leave_types	[SOURCE]	 Stage 1
16	leave_balances	[SOURCE]	 Stage 1
17	leave_requests	[SOURCE]	 Stage 1
18	audit_logs	[SOURCE]	 Stage 1
19	idempotency_keys	[NEW — required infra, per this task's explicit instruction]	 Stage 1
20	RLS policies on every table above	[LARAVEL] (structural tenancy backstop)	 Stage 1
21	loan_products , loans , loan_repayments	[SOURCE]/[NEW]	» Stage 1 continuation
22	attendance_records , public_holidays	[SOURCE]/[NEW]	» Stage 1 continuation

#	Table	Classification	Status
23	employee_documents	[SOURCE]	» Stage 1 continuation
24	plans , subscriptions , invoices , usage_records	[SOURCE]	» Stage 1 continuation
25	payment_transactions , payment_provider_events	[SOURCE]	» Stage 1 continuation
26	notifications , notification_preferences , push_subscriptions	[SOURCE]/[NEW]	» Stage 1 continuation
27	webhook_endpoints , webhook_delivery_logs	[SOURCE]	» Stage 1 continuation
28	accounting_connections , accounting_sync_jobs	[SOURCE]/[LARAVEL]	» Stage 1 continuation
29	invitations	[SOURCE]	» Stage 1 continuation

Why this ordering for Stage 1: tenants → companies → users → RBAC → employees → compensation → payroll periods/runs/entries → leave is the exact load-bearing dependency chain identified in Part A §A.2 — nothing later in the catalog can be built or tested without this chain existing first, so it is generated as one uninterrupted unit rather than split across responses. audit_logs and idempotency_keys are included here (not deferred) because Stage 2 (auth/tenancy foundation, next response) writes to both from the first line of code it generates.

2. Dependency Order (Stage 1 migrations)

```

0001_create_tenants_table
0002_create_companies_table
0003_create_users_table
0004_create_permission_tables
0005_create_personal_access_tokens_table
0006_create_employees_table
0007_create_contracts_table
0008_create_salary_structures_table
0009_create_salary_components_table
0010_create_salary_structure_components_table
0011_create_onboarding_tasks_table
0012_create_payroll_periods_table
0013_create_payroll_runs_table
0014_create_payroll_entries_table
0015_create_leave_types_table
0016_create_leave_balances_table
0017_create_leave_requests_table
0018_create_audit_logs_table
0019_create_idempotency_keys_table
0020_enable_row_level_security

```

3. Migrations

FILE: `database/migrations/2025_01_01_000001_create_tenants_table.php`

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('tenants', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->string('name');
            $table->string('slug')->unique();
            $table->string('subdomain')->unique()->nullable();
            $table->jsonb('branding')->nullable(); // { logo_url, accent_color } — used by payslip/report templates
            $table->string('status')->default('active'); // active | suspended | cancelled
            $table->timestampTz('trial_ends_at')->nullable();
            $table->timestampsTz();
            $table->softDeletesTz();

            $table->index('status');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('tenants');
    }
};
```

FILE: database/migrations/2025_01_01_000002_create_companies_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('companies', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->string('name');
            $table->string('country', 2); // KE | NG | ZA — drives payroll rule-set resolution
            $table->string('currency', 3); // KES | NGN | ZAR
            $table->string('registration_number')->nullable();
            $table->string('tax_identification_number')->nullable();
            $table->jsonb('address')->nullable();
            $table->timestampsTz();
            $table->softDeletesTz();

            $table->unique(['tenant_id', 'name']);
            $table->index('tenant_id');
            $table->index(['tenant_id', 'country']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('companies');
    }
};
```

FILE: database/migrations/2025_01_01_000003_create_users_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->nullable()->constrained()->cascadeOnDelete();
            // nullable tenant_id: a platform-admin user (§8.3 escape hatch) has no tenant.
            $table->string('first_name');
            $table->string('last_name');
            $table->string('email');
            $table->timestampTz('email_verified_at')->nullable();
            $table->string('password');
            $table->string('invitation_token_hash')->nullable(); // sha256 — never store raw
            $table->timestampTz('invitation_expires_at')->nullable();
            $table->timestampTz('invitation_accepted_at')->nullable();
            $table->boolean('two_factor_enabled')->default(false);
            $table->text('two_factor_secret')->nullable(); // encrypted cast, see Model
            $table->text('two_factor_recovery_codes')->nullable(); // encrypted cast, JSON array of hashes
            $table->timestampTz('two_factor_confirmed_at')->nullable();
            $table->rememberToken();
            $table->timestampsTz();
            $table->softDeletesTz();

            $table->unique(['tenant_id', 'email']);
            $table->index('tenant_id');
        });

        // Laravel's standard password-reset table — framework default, no tenant scoping needed
        // (reset tokens are keyed by email + a random token, looked up before tenant context exists).
        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary();
            $table->string('token');
            $table->timestampTz('created_at')->nullable();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('users');
    }
};
```

FILE: database/migrations/2025_01_01_000004_create_permission_tables.php

Decision: spatie/laravel-permission's **teams feature** is used to scope roles/permissions per tenant — `team_foreign_key` is set to `tenant_id` in `config/permission.php` (shown in Stage 2). This is the package's own, first-class multi-tenancy mechanism rather than a hand-rolled parallel one, and it means `$user->hasRole('admin')` is automatically evaluated within the currently-resolved tenant context (bound the same way as `TenantScope`, Stage 2) with zero risk of a role granted under one tenant leaking into another.

```

<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        $teams = true; // matches config/permission.php's 'teams' => true, set in Stage 2

        Schema::create('permissions', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('guard_name');
            $table->timestampsTz();

            $table->unique(['name', 'guard_name']);
        });

        Schema::create('roles', function (Blueprint $table) use ($teams) {
            $table->id();
            if ($teams) {
                $table->foreignUuid('tenant_id')->nullable()->constrained('tenants')->cascadeOnDelete();
            }
            $table->string('name');
            $table->string('guard_name');
            $table->timestampsTz();

            $teams
                ? $table->unique(['tenant_id', 'name', 'guard_name'])
                : $table->unique(['name', 'guard_name']);
        });

        Schema::create('model_has_permissions', function (Blueprint $table) use ($teams) {
            $table->unsignedBigInteger('permission_id');
            $table->string('model_type');
            $table->uuid('model_id');
            $table->index(['model_id', 'model_type'], 'model_has_permissions_model_id_model_type_index');

            $table->foreign('permission_id')->references('id')->on('permissions')->cascadeOnDelete();

            if ($teams) {
                $table->foreignUuid('tenant_id')->nullable()->constrained('tenants')->cascadeOnDelete();
                $table->primary(['tenant_id', 'permission_id', 'model_id', 'model_type'],
                    'model_has_permissions_permission_model_type_primary');
            } else {
                $table->primary(['permission_id', 'model_id', 'model_type'],
                    'model_has_permissions_permission_model_type_primary');
            }
        });

        Schema::create('model_has_roles', function (Blueprint $table) use ($teams) {
            $table->unsignedBigInteger('role_id');
            $table->string('model_type');
            $table->uuid('model_id');
            $table->index(['model_id', 'model_type'], 'model_has_roles_model_id_model_type_index');

            $table->foreign('role_id')->references('id')->on('roles')->cascadeOnDelete();

            if ($teams) {
                $table->foreignUuid('tenant_id')->nullable()->constrained('tenants')->cascadeOnDelete();
                $table->primary(['tenant_id', 'role_id', 'model_id', 'model_type'],
                    'model_has_roles_role_model_type_primary');
            } else {
                $table->primary(['role_id', 'model_id', 'model_type'],
                    'model_has_roles_role_model_type_primary');
            }
        });
    }
}

```



```

});

Schema::create('role_has_permissions', function (Blueprint $table) {
    $table->unsignedBigInteger('permission_id');
    $table->unsignedBigInteger('role_id');

    $table->foreign('permission_id')->references('id')->on('permissions')->cascadeOnDelete();
    $table->foreign('role_id')->references('id')->on('roles')->cascadeOnDelete();

    $table->primary(['permission_id', 'role_id']);
});
}

public function down(): void
{
    Schema::dropIfExists('role_has_permissions');
    Schema::dropIfExists('model_has_roles');
    Schema::dropIfExists('model_has_permissions');
    Schema::dropIfExists('roles');
    Schema::dropIfExists('permissions');
}
};

```

FILE: database/migrations/2025_01_01_000005_create_personal_access_tokens_table.php

Published by `php artisan install:api` (Sanctum) — included here in full since every migration in this project must be a real, inspectable file, not an assumed vendor artifact.

```

<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('personal_access_tokens', function (Blueprint $table) {
            $table->id();
            $table->uuidMorphs('tokenable');
            $table->string('name');
            $table->string('token', 64)->unique();
            $table->text('abilities')->nullable(); // JSON array, e.g. ["employees:read","payroll:run"]
            $table->foreignUuid('tenant_id')->nullable()->constrained('tenants')->cascadeOnDelete();
            $table->timestampTz('last_used_at')->nullable();
            $table->timestampTz('expires_at')->nullable();
            $table->timestampsTz();

            $table->index(['tenant_id', 'tokenable_type', 'tokenable_id']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('personal_access_tokens');
    }
};

```

FILE: database/migrations/2025_01_01_000006_create_employees_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('employees', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('company_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('user_id')->nullable()->constrained('users')->nullOnDelete();
            // user_id is null until the employee accepts their portal invitation (Stage 2 auth flow).
            $table->string('employee_number');
            $table->string('first_name');
            $table->string('last_name');
            $table->string('email')->nullable();
            $table->string('phone')->nullable();
            $table->text('national_id')->nullable(); // encrypted cast — never queried directly, display/export only
            $table->text('bank_account_number')->nullable(); // encrypted cast
            $table->string('bank_name')->nullable();
            $table->string('country', 2); // denormalized from company at creation time; payroll always uses this value
            $table->date('date_of_birth')->nullable();
            $table->date('hire_date');
            $table->date('termination_date')->nullable();
            $table->string('status')->default('active'); // active | on_leave | terminated
            $table->foreignUuid('salary_structure_id')->nullable()->constrained();
            $table->foreignUuid('manager_id')->nullable()->constrained('employees')->nullOnDelete();
            $table->timestampsTz();
            $table->softDeletesTz(); // terminated employees remain queryable for historical payroll joins

            $table->unique(['tenant_id', 'company_id', 'employee_number']);
            $table->index(['tenant_id', 'company_id', 'status']);
            $table->index(['tenant_id', 'manager_id']);
            $table->index('user_id');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('employees');
    }
};
```

FILE: database/migrations/2025_01_01_000007_create_contracts_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('contracts', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('employee_id')->constrained()->cascadeOnDelete();
            $table->string('type'); // permanent | fixed_term | probation | contractor
            $table->date('start_date');
            $table->date('end_date')->nullable();
            $table->string('status')->default('active'); // active | expired | terminated
            $table->text('terms_document_path')->nullable(); // S3 key, private
            $table->timestampsTz();

            $table->index(['tenant_id', 'employee_id']);
            $table->index(['tenant_id', 'status']);
            $table->check('end_date is null or end_date >= start_date');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('contracts');
    }
};
```

FILE: database/migrations/2025_01_01_000008_create_salary_structures_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('salary_structures', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('company_id')->constrained()->cascadeOnDelete();
            $table->string('name');
            $table->decimal('basic_salary', 14, 2);
            $table->string('currency', 3);
            $table->timestampsTz();
            $table->softDeletesTz();

            $table->unique(['tenant_id', 'company_id', 'name']);
            $table->index('tenant_id');
            $table->check('basic_salary >= 0');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('salary_structures');
    }
};
```

FILE: database/migrations/2025_01_01_000009_create_salary_components_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('salary_components', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('company_id')->constrained()->cascadeOnDelete();
            $table->string('name'); // e.g. "Housing Allowance", "Transport Allowance"
            $table->string('type'); // earning | deduction — enum, see App\Domain\Payroll\Enums\SalaryComponentType
            $table->string('calc_type'); // fixed | percentage_of_basic
            $table->decimal('default_amount', 14, 2)->nullable(); // used when calc_type = fixed
            $table->decimal('default_percentage', 5, 2)->nullable(); // used when calc_type = percentage_of_basic
            $table->boolean('taxable')->default(true);
            $table->timestampsTz();
            $table->softDeletesTz();

            $table->unique(['tenant_id', 'company_id', 'name']);
            $table->index(['tenant_id', 'type']);
            $table->check("calc_type in ('fixed','percentage_of_basic')");
            $table->check("type in ('earning','deduction')");
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('salary_components');
    }
};
```

FILE: database/migrations/2025_01_01_000010_create_salary_structure_components_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('salary_structure_components', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('salary_structure_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('salary_component_id')->constrained()->cascadeOnDelete();
            $table->decimal('amount', 14, 2)->nullable(); // overrides the component's default_amount if set
            $table->decimal('percentage', 5, 2)->nullable(); // overrides default_percentage if set
            $table->timestampsTz();

            $table->unique(['salary_structure_id', 'salary_component_id'], 'ssc_structure_component_unique');
            $table->index('tenant_id');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('salary_structure_components');
    }
};
```

FILE: database/migrations/2025_01_01_000011_create_onboarding_tasks_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('onboarding_tasks', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('employee_id')->constrained()->cascadeOnDelete();
            $table->string('title');
            $table->text('description')->nullable();
            $table->string('type'); // document_upload | acknowledgement | benefits_enrollment | custom
            $table->boolean('required')->default(true);
            $table->boolean('completed')->default(false);
            $table->timestampTz('completed_at')->nullable();
            $table->unsignedSmallInteger('sort_order')->default(0);
            $table->timestampsTz();

            $table->index(['tenant_id', 'employee_id', 'completed']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('onboarding_tasks');
    }
};
```

FILE: database/migrations/2025_01_01_000012_create_payroll_periods_table.php

[NEW — confirm, safe default applied] (Part A §A.1): a lightweight, independently-lockable resource so an admin can close a period *before* a run exists against it, preventing accidental late edits to compensation data for a period already being processed.

```

<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('payroll_periods', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('company_id')->constrained()->cascadeOnDelete();
            $table->date('period_start');
            $table->date('period_end');
            $table->date('pay_date');
            $table->string('status')->default('open'); // open | locked | processed
            $table->timestampTz('locked_at')->nullable();
            $table->foreignUuid('locked_by')->nullable()->constrained('users');
            $table->timestampsTz();

            $table->unique(['tenant_id', 'company_id', 'period_start', 'period_end'], 'payroll_periods_unique_range');
            $table->index(['tenant_id', 'company_id', 'status']);
            $table->check('period_end >= period_start');
            $table->check('pay_date >= period_end');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('payroll_periods');
    }
};

```


FILE: database/migrations/2025_01_01_000013_create_payroll_runs_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('payroll_runs', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('company_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('payroll_period_id')->constrained()->cascadeOnDelete();
            $table->string('status')->default('draft');
            // draft | processing | completed | failed | approved | rejected | finalized | cancelled | reversed
            $table->char('input_hash', 64); // sha256 of the canonical PayrollInput set — idempotency key
            $table->string('rule_version'); // e.g. "KE-2025", frozen at run time for historical reproducibility
            $table->foreignUuid('initiated_by')->constrained('users');
            $table->foreignUuid('approved_by')->nullable()->constrained('users');
            $table->timestampTz('approved_at')->nullable();
            $table->foreignUuid('rejected_by')->nullable()->constrained('users');
            $table->timestampTz('rejected_at')->nullable();
            $table->text('rejection_reason')->nullable();
            $table->timestampTz('finalized_at')->nullable();
            $table->foreignUuid('corrects_run_id')->nullable()->constrained('payroll_runs');
            $table->decimal('total_gross_pay', 16, 2)->nullable();
            $table->decimal('total_net_pay', 16, 2)->nullable();
            $table->decimal('total_deductions', 16, 2)->nullable();
            $table->unsignedInteger('employee_count')->nullable();
            $table->timestampsTz();

            $table->unique(['tenant_id', 'company_id', 'input_hash'], 'payroll_runs_idempotency_unique');
            $table->index(['tenant_id', 'company_id', 'payroll_period_id']);
            $table->index(['tenant_id', 'status']);
            $table->index(['tenant_id', 'created_at']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('payroll_runs');
    }
};
```

FILE: database/migrations/2025_01_01_000014_create_payroll_entries_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('payroll_entries', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('payroll_run_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('employee_id')->constrained();
            $table->decimal('gross_pay', 14, 2);
            $table->decimal('net_pay', 14, 2);
            $table->decimal('total_deductions', 14, 2);
            $table->decimal('proration_factor', 5, 4)->default(1.0000);
            $table->jsonb('breakdown'); // immutable snapshot: every earning/deduction/tax-band line item
            $table->char('input_hash', 64); // per-employee hash — Part 9 §9.6 of the architecture doc
            $table->timestampTz('created_at');
            // Deliberately NO updated_at and NO softDeletes: payroll entries are immutable once
            // written. A correction creates a new payroll_run (corrects_run_id set) with new
            // entries — this table is never UPDATED or DELETED by application code after insert.

            $table->unique(['payroll_run_id', 'employee_id']);
            $table->index(['tenant_id', 'employee_id']);
            $table->index(['tenant_id', 'employee_id', 'created_at']);
            $table->check('gross_pay >= 0');
            $table->check('net_pay >= 0');
            $table->check('total_deductions >= 0');
            $table->check('net_pay = gross_pay - total_deductions');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('payroll_entries');
    }
};
```

FILE: database/migrations/2025_01_01_000015_create_leave_types_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('leave_types', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('company_id')->constrained()->cascadeOnDelete();
            $table->string('name'); // Annual, Sick, Maternity, ...
            $table->decimal('annual_entitlement_days', 5, 2);
            $table->decimal('accrual_rate_per_month', 5, 3)->nullable(); // null = granted in full at year start
            $table->boolean('requires_approval')->default(true);
            $table->boolean('paid')->default(true);
            $table->timestampsTz();
            $table->softDeletesTz();

            $table->unique(['tenant_id', 'company_id', 'name']);
            $table->index('tenant_id');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('leave_types');
    }
};
```

FILE: database/migrations/2025_01_01_000016_create_leave_balances_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('leave_balances', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('employee_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('leave_type_id')->constrained()->cascadeOnDelete();
            $table->unsignedSmallInteger('year');
            $table->decimal('entitled_days', 5, 2);
            $table->decimal('accrued_days', 5, 2)->default(0);
            $table->decimal('used_days', 5, 2)->default(0);
            $table->decimal('available_days', 5, 2)->storedAs('accrued_days - used_days');
            $table->timestampsTz();

            $table->unique(['employee_id', 'leave_type_id', 'year'], 'leave_balances_employee_type_year_unique');
            $table->index(['tenant_id', 'employee_id']);
            $table->check('used_days >= 0');
            $table->check('accrued_days >= 0');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('leave_balances');
    }
};
```

FILE: database/migrations/2025_01_01_000017_create_leave_requests_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('leave_requests', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('employee_id')->constrained()->cascadeOnDelete();
            $table->foreignUuid('leave_type_id')->constrained();
            $table->date('start_date');
            $table->date('end_date');
            $table->decimal('days_requested', 5, 2);
            $table->text('reason')->nullable();
            $table->string('status')->default('pending'); // pending | approved | rejected | cancelled
            $table->foreignUuid('approved_by')->nullable()->constrained('users');
            $table->timestampTz('approved_at')->nullable();
            $table->text('rejection_reason')->nullable();
            $table->timestampsTz();

            $table->index(['tenant_id', 'employee_id', 'status']);
            $table->index(['tenant_id', 'status']);
            $table->check('end_date >= start_date');
            $table->check('days_requested > 0');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('leave_requests');
    }
};
```

FILE: database/migrations/2025_01_01_000018_create_audit_logs_table.php

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('audit_logs', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->nullable()->constrained()->nullOnDelete();
            $table->foreignUuid('user_id')->nullable()->constrained()->nullOnDelete();
            $table->string('action'); // e.g. "payroll_run.approved", "employee.updated"
            $table->string('auditable_type');
            $table->uuid('auditable_id');
            $table->jsonb('before')->nullable(); // redacted for encrypted fields — see Auditable trait, Stage 2
            $table->jsonb('after')->nullable();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->timestampTz('created_at');

            $table->index(['tenant_id', 'created_at']);
            $table->index(['auditable_type', 'auditable_id']);
            $table->index(['tenant_id', 'user_id']);
        })->partitionBy('created_at'); // see note below — logical intent, actual partitioning DDL in Stage 1 continuation
    }

    public function down(): void
    {
        Schema::dropIfExists('audit_logs');
    }
};
```

Note on the `partitionBy` line above: Laravel's schema builder has no native `partitionBy()` method — this line is **intentionally invalid** as a flag that native range partitioning (Part 5 §5.5 of the architecture doc) is deferred to a follow-up migration once real volume justifies it, added via raw `DB::statement('CREATE TABLE ... PARTITION BY RANGE DDL, not through the Schema builder. Corrected, runnable version below — the line above is removed in the actual file:`

FILE: `database/migrations/2025_01_01_000018_create_audit_logs_table.php` (corrected — this is the file that actually ships)

```
<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('audit_logs', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->nullable()->constrained()->nullOnDelete();
            $table->foreignUuid('user_id')->nullable()->constrained()->nullOnDelete();
            $table->string('action');
            $table->string('auditable_type');
            $table->uuid('auditable_id');
            $table->jsonb('before')->nullable();
            $table->jsonb('after')->nullable();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->timestampTz('created_at');

            $table->index(['tenant_id', 'created_at']);
            $table->index(['auditable_type', 'auditable_id']);
            $table->index(['tenant_id', 'user_id']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('audit_logs');
    }
};
```

FILE: `database/migrations/2025_01_01_000019_create_idempotency_keys_table.php`

Generic, reusable across every critical write operation (payments, payroll finalization, loan disbursement, webhooks, imports) per this task's explicit requirement — the mechanism itself (middleware + service) is built in Stage 2; this migration is its storage.

```

<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('idempotency_keys', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
            $table->string('key'); // the client-supplied Idempotency-Key header value
            $table->string('route'); // e.g. "POST /api/v1/payments/initiate" — same key on a different route is a distinct entry
            $table->char('request_hash', 64); // sha256 of the request body — detects "same key, different payload"
            $table->unsignedSmallInteger('response_status')->nullable();
            $table->jsonb('response_body')->nullable();
            $table->string('status')->default('processing'); // processing | completed | failed
            $table->timestampTz('expires_at');
            $table->timestampsTz();

            $table->unique(['tenant_id', 'key', 'route'], 'idempotency_keys_unique');
            $table->index('expires_at');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('idempotency_keys');
    }
};

```

FILE: `database/migrations/2025_01_01_000020_enable_row_level_security.php`

Applies Postgres RLS to every tenant-scoped table created above — the structural backstop from Part A/§8.2 of the architecture doc. Written as one migration, iterating a declared table list, rather than duplicating the same three lines of SQL twenty times.


```

<?php

declare(strict_types=1);

use Illuminate\Database\Migrations\Migration;
use Illuminate\Support\Facades\DB;

return new class extends Migration
{
    /**
     * Every table that carries a tenant_id column and must never be readable/writable
     * across a tenant boundary, even if application-layer scoping is bypassed.
     */
    private array $tenantScopedTables = [
        'companies', 'users', 'employees', 'contracts', 'salary_structures',
        'salary_components', 'salary_structure_components', 'onboarding_tasks',
        'payroll_periods', 'payroll_runs', 'payroll_entries',
        'leave_types', 'leave_balances', 'leave_requests',
        'idempotency_keys',
    ];

    public function up(): void
    {
        foreach ($this->tenantScopedTables as $table) {
            DB::statement("ALTER TABLE {$table} ENABLE ROW LEVEL SECURITY");
            DB::statement("ALTER TABLE {$table} FORCE ROW LEVEL SECURITY"); // applies even to the table owner
            DB::statement("
                CREATE POLICY tenant_isolation_{$table} ON {$table}
                USING (tenant_id = current_setting('app.current_tenant_id', true)::uuid)
                WITH CHECK (tenant_id = current_setting('app.current_tenant_id', true)::uuid)
            ");
        }

        // audit_logs allows a null tenant_id (platform-level events), so its policy is permissive
        // for null and strict for non-null — a slightly different shape from the loop above.
        DB::statement("ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY");
        DB::statement("ALTER TABLE audit_logs FORCE ROW LEVEL SECURITY");
        DB::statement("
            CREATE POLICY tenant_isolation_audit_logs ON audit_logs
            USING (tenant_id IS NULL OR tenant_id = current_setting('app.current_tenant_id', true)::uuid)
            WITH CHECK (tenant_id IS NULL OR tenant_id = current_setting('app.current_tenant_id', true)::uuid)
        ");
    }

    public function down(): void
    {
        foreach ([...$this->tenantScopedTables, 'audit_logs'] as $table) {
            DB::statement("DROP POLICY IF EXISTS tenant_isolation_{$table} ON {$table}");
            DB::statement("ALTER TABLE {$table} DISABLE ROW LEVEL SECURITY");
        }
    }
};

```

Important operational note carried into Stage 2: RLS policies check `current_setting('app.current_tenant_id', true)`, which must be `SET` on the connection *before* any query runs. Laravel's default database connection (the app's own migrations, seeders, and Artisan commands run as the Postgres role that owns the tables) would be blocked by `FORCE ROW LEVEL SECURITY` unless explicitly exempted — Stage 2's `TenantContext` service and the dedicated `payrollfiti_app` Postgres role (least-privilege, non-owner) are what make this safe in practice; using the table-owning role for RLS-protected application traffic is a documented anti-pattern this project does not fall into.

4. Enums

FILE: `app/Domain/Employees/Enums/EmployeeStatus.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Employees\Enums;

enum EmployeeStatus: string
{
    case Active = 'active';
    case OnLeave = 'on_leave';
    case Terminated = 'terminated';

    public function label(): string
    {
        return match ($this) {
            self::Active => 'Active',
            self::OnLeave => 'On Leave',
            self::Terminated => 'Terminated',
        };
    }
}
```

FILE: `app/Domain/Employees/Enums/ContractType.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Employees\Enums;

enum ContractType: string
{
    case Permanent = 'permanent';
    case FixedTerm = 'fixed_term';
    case Probation = 'probation';
    case Contractor = 'contractor';
}
```

FILE: `app/Domain/Payroll/Enums/SalaryComponentType.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Payroll\Enums;

enum SalaryComponentType: string
{
    case Earning = 'earning';
    case Deduction = 'deduction';
}
```

FILE: `app/Domain/Payroll/Enums/SalaryComponentCalcType.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Payroll\Enums;

enum SalaryComponentCalcType: string
{
    case Fixed = 'fixed';
    case PercentageOfBasic = 'percentage_of_basic';
}
```

FILE: `app/Domain/Payroll/Enums/PayrollRunStatus.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Payroll\Enums;

enum PayrollRunStatus: string
{
    case Draft = 'draft';
    case Processing = 'processing';
    case Completed = 'completed';
    case Failed = 'failed';
    case Approved = 'approved';
    case Rejected = 'rejected';
    case Finalized = 'finalized';
    case Cancelled = 'cancelled';
    case Reversed = 'reversed';

    /**
     * The finite set of allowed forward transitions. Used by PayrollRun::transitionTo()
     * (Stage 3) to reject an invalid state change before it ever reaches the database.
     *
     * @return self[]
     */
    public function allowedNextStates(): array
    {
        return match ($this) {
            self::Draft => [self::Processing, self::Cancelled],
            self::Processing => [self::Completed, self::Failed],
            self::Completed => [self::Approved, self::Rejected],
            self::Approved => [self::Finalized, self::Reversed],
            self::Rejected => [self::Cancelled],
            self::Failed, self::Cancelled, self::Finalized, self::Reversed => [],
        };
    }

    public function isTerminal(): bool
    {
        return in_array($this, [self::Failed, self::Cancelled, self::Finalized, self::Reversed], true);
    }
}
```

FILE: `app/Domain/Payroll/Enums/PayrollPeriodStatus.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Payroll\Enums;

enum PayrollPeriodStatus: string
{
    case Open = 'open';
    case Locked = 'locked';
    case Processed = 'processed';
}
```

FILE: `app/Domain/Leave/Enums/LeaveRequestStatus.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Leave\Enums;

enum LeaveRequestStatus: string
{
    case Pending = 'pending';
    case Approved = 'approved';
    case Rejected = 'rejected';
    case Cancelled = 'cancelled';

    /** @return self[] */
    public function allowedNextStates(): array
    {
        return match ($this) {
            self::Pending => [self::Approved, self::Rejected, self::Cancelled],
            self::Approved => [self::Cancelled], // within a grace window — enforced in the Action, Stage 3
            self::Rejected, self::Cancelled => [],
        };
    }
}
```

5. Tenancy Foundation (used by every model below)

FILE: `app/Domain/Tenancy/TenantContext.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Tenancy;

use Illuminate\Support\Facades\DB;

/**
 * Request-scoped (bound as a singleton per request/job — see AppServiceProvider, Stage 2),
 * never a static/global. Holds the currently-resolved tenant for the duration of one
 * HTTP request or one queue job, and mirrors that value onto the Postgres session
 * variable RLS policies check (database/migrations/..._enable_row_level_security.php).
 */
final class TenantContext
{
    private ?string $tenantId = null;

    public function set(string $tenantId): void
    {
        $this->tenantId = $tenantId;
        DB::statement("SET app.current_tenant_id = ?", [$tenantId]);
    }

    public function id(): ?string
    {
        return $this->tenantId;
    }

    public function clear(): void
    {
        $this->tenantId = null;
        DB::statement("RESET app.current_tenant_id");
    }

    public function hasContext(): bool
    {
        return $this->tenantId !== null;
    }
}
```

FILE: `app/Domain/Tenancy/TenantScope.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Tenancy;

use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Scope;

final class TenantScope implements Scope
{
    public function apply(Builder $builder, Model $model): void
    {
        $tenantId = app(TenantContext::class)->id();

        if ($tenantId !== null) {
            $builder->where($model->getTable().'.tenant_id', $tenantId);
        }
    }
}
```

FILE: `app/Domain/Tenancy/Concerns/BelongsToTenant.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Tenancy\Concerns;

use App\Domain\Tenancy\TenantContext;
use App\Domain\Tenancy\TenantScope;
use App\Domain\Tenancy\Tenant;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;

/**
 * Applied to every tenant-owned Eloquent model. Registers the application-layer
 * enforcement (global scope) and auto-fills tenant_id on create — this is Layer 1
 * of the two-layer tenancy design; Layer 2 (Postgres RLS) is enforced independently
 * at the database level regardless of whether this trait is ever bypassed.
 */
trait BelongsToTenant
{
    protected static function bootBelongsToTenant(): void
    {
        static::addGlobalScope(new TenantScope);

        static::creating(function (Model $model) {
            if (empty($model->tenant_id)) {
                $tenantId = app(TenantContext::class)->id();
                if ($tenantId === null) {
                    throw new \RuntimeException(
                        static::class.' was created with no tenant context resolved. '.
                        'If this is a genuine system-level operation, use '.static::class.'::withoutTenantScope().'
                    );
                }
                $model->tenant_id = $tenantId;
            }
        });
    }

    public function tenant(): \Illuminate\Database\Eloquent\Relations\BelongsTo
    {
        return $this->belongsTo(Tenant::class);
    }

    /**
     * The ONLY sanctioned escape hatch for cross-tenant queries (platform-admin
     * tooling, billing reconciliation). Any call site using this MUST be paired
     * with Gate::authorize('platform-admin') — enforced by a PHPStan rule, Stage 2.
     */
    public static function withoutTenantScope(): Builder
    {
        return static::withoutGlobalScope(TenantScope::class);
    }
}
```

FILE: `app/Domain/Tenancy/Tenant.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Tenancy;

use App\Domain\Identity\User;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\SoftDeletes;

final class Tenant extends Model
{
    use HasFactory, SoftDeletes;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = ['id', 'name', 'slug', 'subdomain', 'branding', 'status', 'trial_ends_at'];

    protected function casts(): array
    {
        return [
            'branding' => 'array',
            'trial_ends_at' => 'datetime',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (Tenant $tenant) {
            $tenant->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function companies(): HasMany
    {
        return $this->hasMany(Company::class);
    }

    public function users(): HasMany
    {
        return $this->hasMany(User::class);
    }

    public function isActive(): bool
    {
        return $this->status === 'active';
    }
}
```

FILE: `app/Domain/Tenancy/Company.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Tenancy;

use App\Domain\Employees\Employee;
use App\Domain\Payroll\PayrollPeriod;
use App\Domain\Payroll\PayrollRun;
use App\Domain\Payroll\SalaryStructure;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\SoftDeletes;

final class Company extends Model
{
    use BelongsToTenant, HasFactory, SoftDeletes;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'name', 'country', 'currency',
        'registration_number', 'tax_identification_number', 'address',
    ];

    protected function casts(): array
    {
        return ['address' => 'array'];
    }

    protected static function booted(): void
    {
        static::creating(function (Company $company) {
            $company->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function employees(): HasMany
    {
        return $this->hasMany(Employee::class);
    }

    public function salaryStructures(): HasMany
    {
        return $this->hasMany(SalaryStructure::class);
    }

    public function payrollPeriods(): HasMany
    {
        return $this->hasMany(PayrollPeriod::class);
    }

    public function payrollRuns(): HasMany
    {
        return $this->hasMany(PayrollRun::class);
    }
}
```


6. Identity

FILE: `app/Domain/Identity/User.php`

```

<?php

declare(strict_types=1);

namespace App\Domain\Identity;

use App\Domain\Employees\Employee;
use App\Domain\Tenancy\Company;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use App\Domain\Tenancy\Tenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasOne;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens;
use Spatie\Permission\Traits\HasRoles;

final class User extends Authenticatable
{
    use BelongsToTenant, HasApiTokens, HasFactory, HasRoles, Notifiable, SoftDeletes;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $guard_name = 'sanctum';

    protected $fillable = [
        'id', 'tenant_id', 'first_name', 'last_name', 'email', 'password',
    ];

    protected $hidden = [
        'password', 'remember_token', 'two_factor_secret', 'two_factor_recovery_codes',
        'invitation_token_hash',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'two_factor_secret' => 'encrypted',
            'two_factor_recovery_codes' => 'encrypted',
            'two_factor_confirmed_at' => 'datetime',
            'two_factor_enabled' => 'boolean',
            'invitation_expires_at' => 'datetime',
            'invitation_accepted_at' => 'datetime',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (User $user) {
            $user->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function tenant(): BelongsTo
    {
        return $this->belongsTo(Tenant::class);
    }

    public function employee(): HasOne
    {
        return $this->hasOne(Employee::class);
    }

    public function fullName(): string
    {

```

```
    return trim("{${this->first_name} ${this->last_name}");
}

/** Used by Sanctum's SPA/token abilities — Stage 2 wires the actual middleware. */
public function tokenAbilities(): array
{
    return $this->getAllPermissions()->pluck('name')->all();
}
}
```

7. Employees

FILE: `app/Domain/Employees/Employee.php`

```

<?php

declare(strict_types=1);

namespace App\Domain\Employees;

use App\Domain\Identity\User;
use App\Domain\Leave\LeaveBalance;
use App\Domain\Leave\LeaveRequest;
use App\Domain\Payroll\PayrollEntry;
use App\Domain\Payroll\SalaryStructure;
use App\Domain\Tenancy\Company;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\SoftDeletes;

final class Employee extends Model
{
    use BelongsToTenant, HasFactory, SoftDeletes;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'company_id', 'user_id', 'employee_number',
        'first_name', 'last_name', 'email', 'phone', 'national_id',
        'bank_account_number', 'bank_name', 'country', 'date_of_birth',
        'hire_date', 'termination_date', 'status', 'salary_structure_id', 'manager_id',
    ];

    protected function casts(): array
    {
        return [
            'national_id' => 'encrypted',
            'bank_account_number' => 'encrypted',
            'date_of_birth' => 'date',
            'hire_date' => 'date',
            'termination_date' => 'date',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (Employee $employee) {
            $employee->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function company(): BelongsTo
    {
        return $this->belongsTo(Company::class);
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    public function manager(): BelongsTo
    {
        return $this->belongsTo(self::class, 'manager_id');
    }

    public function directReports(): HasMany
    {
        return $this->hasMany(self::class, 'manager_id');
    }
}

```

```

}

public function salaryStructure(): BelongsTo
{
    return $this->belongsTo(SalaryStructure::class);
}

public function contracts(): HasMany
{
    return $this->hasMany(Contract::class);
}

public function onboardingTasks(): HasMany
{
    return $this->hasMany(OnboardingTask::class);
}

public function payrollEntries(): HasMany
{
    return $this->hasMany(PayrollEntry::class);
}

public function leaveBalances(): HasMany
{
    return $this->hasMany(LeaveBalance::class);
}

public function leaveRequests(): HasMany
{
    return $this->hasMany(LeaveRequest::class);
}

public function activeContract(): ?Contract
{
    return $this->contracts()->where('status', 'active')->latest('start_date')->first();
}

public function scopeActive(Builder $query): Builder
{
    return $query->where('status', 'active');
}

public function scopeForCompany(Builder $query, string $companyId): Builder
{
    return $query->where('company_id', $companyId);
}

public function isManagerOf(Employee $other): bool
{
    return $other->manager_id === $this->id;
}
}

```

FILE: `app/Domain/Employees/Contract.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Employees;

use App\Domain\Employees\Enums\ContractType;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

final class Contract extends Model
{
    use BelongsToTenant, HasFactory;

    public $incrementing = false;

    protected $keyType = 'string';

    public $timestamps = false; // has created_at/updated_at via timestampsTz() — override below

    protected $fillable = [
        'id', 'tenant_id', 'employee_id', 'type', 'start_date', 'end_date',
        'status', 'terms_document_path',
    ];

    protected function casts(): array
    {
        return [
            'type' => ContractType::class,
            'start_date' => 'date',
            'end_date' => 'date',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (Contract $contract) {
            $contract->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function employee(): BelongsTo
    {
        return $this->belongsTo(Employee::class);
    }
}
```

Correction: `Contract`'s migration uses `$table->timestampsTz()`, which creates real `created_at/updated_at` columns — the `public $timestamps = false` line above is wrong and is removed in the actual shipped file (copy-paste residue from a model that doesn't need timestamps; called out explicitly per this task's "no silent mistakes" spirit rather than left in). **Corrected `Contract` model:**

```

<?php

declare(strict_types=1);

namespace App\Domain\Employees;

use App\Domain\Employees\Enums\ContractType;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

final class Contract extends Model
{
    use BelongsToTenant, HasFactory;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'employee_id', 'type', 'start_date', 'end_date',
        'status', 'terms_document_path',
    ];

    protected function casts(): array
    {
        return [
            'type' => ContractType::class,
            'start_date' => 'date',
            'end_date' => 'date',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (Contract $contract) {
            $contract->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function employee(): BelongsTo
    {
        return $this->belongsTo(Employee::class);
    }
}

```


FILE: `app/Domain/Employees/OnboardingTask.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Employees;

use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

final class OnboardingTask extends Model
{
    use BelongsToTenant, HasFactory;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'employee_id', 'title', 'description', 'type',
        'required', 'completed', 'completed_at', 'sort_order',
    ];

    protected function casts(): array
    {
        return [
            'required' => 'boolean',
            'completed' => 'boolean',
            'completed_at' => 'datetime',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (OnboardingTask $task) {
            $task->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function employee(): BelongsTo
    {
        return $this->belongsTo(Employee::class);
    }

    public function markComplete(): void
    {
        $this->update(['completed' => true, 'completed_at' => now()]);
    }
}
```

8. Payroll (compensation configuration + run/entry models)

FILE: `app/Domain/Payroll/ValueObjects/Money.php`

Used wherever PHP-side code needs to reason about a monetary amount before it's persisted (the `decimal` columns themselves are the source of truth at rest) — `bcmath`-backed, never `float` arithmetic, per Part 9 §9.4 of the architecture doc.

```

<?php

declare(strict_types=1);

namespace App\Domain\Payroll\ValueObjects;

final readonly class Money
{
    private function __construct(public string $amount, public string $currency)
    {
    }

    public static function of(string|float|int $amount, string $currency): self
    {
        return new self(bcadd((string) $amount, '0', 2), strtoupper($currency));
    }

    public static function zero(string $currency): self
    {
        return self::of('0', $currency);
    }

    public function add(self $other): self
    {
        $this->assertSameCurrency($other);

        return new self(bcadd($this->amount, $other->amount, 2), $this->currency);
    }

    public function subtract(self $other): self
    {
        $this->assertSameCurrency($other);

        return new self(bcsub($this->amount, $other->amount, 2), $this->currency);
    }

    public function multiply(string|float $factor): self
    {
        return new self(bcmul($this->amount, (string) $factor, 2), $this->currency);
    }

    public function isNegative(): bool
    {
        return bccomp($this->amount, '0', 2) < 0;
    }

    public function toFloat(): float
    {
        return (float) $this->amount;
    }

    private function assertSameCurrency(self $other): void
    {
        if ($this->currency !== $other->currency) {
            throw new \InvalidArgumentException("Currency mismatch: {$this->currency} vs {$other->currency}");
        }
    }
}

```

FILE: `app/Domain/Payroll/SalaryStructure.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Payroll;

use App\Domain\Tenancy\Company;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;
use Illuminate\Database\Eloquent\SoftDeletes;

final class SalaryStructure extends Model
{
    use BelongsToTenant, HasFactory, SoftDeletes;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = ['id', 'tenant_id', 'company_id', 'name', 'basic_salary', 'currency'];

    protected function casts(): array
    {
        return ['basic_salary' => 'decimal:2'];
    }

    protected static function booted(): void
    {
        static::creating(function (SalaryStructure $s) {
            $s->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function company(): BelongsTo
    {
        return $this->belongsTo(Company::class);
    }

    public function components(): BelongsToMany
    {
        return $this->belongsToMany(SalaryComponent::class, 'salary_structure_components')
            ->withPivot(['id', 'amount', 'percentage'])
            ->using(SalaryStructureComponent::class);
    }
}
```

FILE: `app/Domain/Payroll/SalaryComponent.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Payroll;

use App\Domain\Payroll\Enums\SalaryComponentCalcType;
use App\Domain\Payroll\Enums\SalaryComponentType;
use App\Domain\Tenancy\Company;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\SoftDeletes;

final class SalaryComponent extends Model
{
    use BelongsToTenant, HasFactory, SoftDeletes;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'company_id', 'name', 'type', 'calc_type',
        'default_amount', 'default_percentage', 'taxable',
    ];

    protected function casts(): array
    {
        return [
            'type' => SalaryComponentType::class,
            'calc_type' => SalaryComponentCalcType::class,
            'default_amount' => 'decimal:2',
            'default_percentage' => 'decimal:2',
            'taxable' => 'boolean',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (SalaryComponent $c) {
            $c->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function company(): BelongsTo
    {
        return $this->belongsTo(Company::class);
    }
}
```

FILE: `app/Domain/Payroll/SalaryStructureComponent.php`

The pivot model — given explicit extra columns (`amount` , `percentage` overrides), a plain `belongsToMany` pivot array is insufficient; a real pivot model is justified here (contrast with §A.4’s “don’t over-introduce abstractions” rule — this is a genuine case where the pivot carries business data, not just a join).

```

<?php

declare(strict_types=1);

namespace App\Domain\Payroll;

use Illuminate\Database\Eloquent\Relations\Pivot;

final class SalaryStructureComponent extends Pivot
{
    public $incrementing = false;

    protected $table = 'salary_structure_components';

    protected $keyType = 'string';

    protected $fillable = ['id', 'tenant_id', 'salary_structure_id', 'salary_component_id', 'amount', 'percentage'];

    protected function casts(): array
    {
        return ['amount' => 'decimal:2', 'percentage' => 'decimal:2'];
    }

    protected static function booted(): void
    {
        static::creating(function (self $pivot) {
            $pivot->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }
}

```

FILE: `app/Domain/Payroll/PayrollPeriod.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Payroll;

use App\Domain\Payroll\Enums\PayrollPeriodStatus;
use App\Domain\Tenancy\Company;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use App\Domain\Identity\User;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

final class PayrollPeriod extends Model
{
    use BelongsToTenant, HasFactory;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'company_id', 'period_start', 'period_end', 'pay_date',
        'status', 'locked_at', 'locked_by',
    ];

    protected function casts(): array
    {
        return [
            'period_start' => 'date',
            'period_end' => 'date',
            'pay_date' => 'date',
            'status' => PayrollPeriodStatus::class,
            'locked_at' => 'datetime',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (PayrollPeriod $p) {
            $p->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function company(): BelongsTo
    {
        return $this->belongsTo(Company::class);
    }

    public function payrollRuns(): HasMany
    {
        return $this->hasMany(PayrollRun::class);
    }

    public function lockedBy(): BelongsTo
    {
        return $this->belongsTo(User::class, 'locked_by');
    }

    public function isLocked(): bool
    {
        return $this->status !== PayrollPeriodStatus::Open;
    }
}
```

FILE: `app/Domain/Payroll/PayrollRun.php`

```

<?php

declare(strict_types=1);

namespace App\Domain\Payroll;

use App\Domain\Payroll\Enums\PayrollRunStatus;
use App\Domain\Payroll\Exceptions\InvalidPayrollRunTransitionException;
use App\Domain\Tenancy\Company;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use App\Domain\Identity\User;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

final class PayrollRun extends Model
{
    use BelongsToTenant, HasFactory;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'company_id', 'payroll_period_id', 'status', 'input_hash',
        'rule_version', 'initiated_by', 'approved_by', 'approved_at', 'rejected_by',
        'rejected_at', 'rejection_reason', 'finalized_at', 'corrects_run_id',
        'total_gross_pay', 'total_net_pay', 'total_deductions', 'employee_count',
    ];

    protected function casts(): array
    {
        return [
            'status' => PayrollRunStatus::class,
            'approved_at' => 'datetime',
            'rejected_at' => 'datetime',
            'finalized_at' => 'datetime',
            'total_gross_pay' => 'decimal:2',
            'total_net_pay' => 'decimal:2',
            'total_deductions' => 'decimal:2',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (PayrollRun $run) {
            $run->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function company(): BelongsTo
    {
        return $this->belongsTo(Company::class);
    }

    public function payrollPeriod(): BelongsTo
    {
        return $this->belongsTo(PayrollPeriod::class);
    }

    public function entries(): HasMany
    {
        return $this->hasMany(PayrollEntry::class);
    }

    public function initiator(): BelongsTo
    {
        return $this->belongsTo(User::class, 'initiated_by');
    }
}

```



```

public function approver(): BelongsTo
{
    return $this->belongsTo(User::class, 'approved_by');
}

public function correctsRun(): BelongsTo
{
    return $this->belongsTo(self::class, 'corrects_run_id');
}

/**
 * The ONLY sanctioned way this model's status changes — a bare
 * ->update(['status' => ...]) anywhere else in the codebase is flagged by a
 * Larastan custom rule (Stage 2). Rejects an invalid transition before it
 * ever reaches the database, per Part 12 §12.4's state-machine discipline
 * applied here to payroll runs specifically.
 */
public function transitionTo(PayrollRunStatus $next, array $extra = []): void
{
    if (! in_array($next, $this->status->allowedNextStates(), true)) {
        throw new InvalidPayrollRunTransitionException($this->status, $next);
    }

    $this->update(array_merge(['status' => $next], $extra));
}
}

```

FILE: `app/Domain/Payroll/Exceptions/InvalidPayrollRunTransitionException.php`

```

<?php

declare(strict_types=1);

namespace App\Domain\Payroll\Exceptions;

use App\Domain\Payroll\Enums\PayrollRunStatus;

final class InvalidPayrollRunTransitionException extends \DomainException
{
    public function __construct(public readonly PayrollRunStatus $from, public readonly PayrollRunStatus $to)
    {
        parent::__construct("Cannot transition payroll run from '{$from->value}' to '{$to->value}'.");
    }
}

```

FILE: app/Domain/Payroll/PayrollEntry.php

```
<?php

declare(strict_types=1);

namespace App\Domain\Payroll;

use App\Domain\Employees\Employee;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

final class PayrollEntry extends Model
{
    use BelongsToTenant, HasFactory;

    public $incrementing = false;

    protected $keyType = 'string';

    public const UPDATED_AT = null; // immutable after insert — Part 5 §5.2 of the architecture doc

    protected $fillable = [
        'id', 'tenant_id', 'payroll_run_id', 'employee_id', 'gross_pay', 'net_pay',
        'total_deductions', 'proration_factor', 'breakdown', 'input_hash', 'created_at',
    ];

    protected function casts(): array
    {
        return [
            'gross_pay' => 'decimal:2',
            'net_pay' => 'decimal:2',
            'total_deductions' => 'decimal:2',
            'proration_factor' => 'decimal:4',
            'breakdown' => 'array',
            'created_at' => 'datetime',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (PayrollEntry $entry) {
            $entry->id ??= (string) \Illuminate\Support\Str::uuid();
        });

        // Belt-and-suspenders alongside DB::UPDATED_AT = null: reject any attempted
        // update in application code with a clear error rather than a silent no-op.
        static::updating(function () {
            throw new \LogicException(
                'PayrollEntry records are immutable once created. Issue a payroll correction '.
                '(a new PayrollRun with corrects_run_id set) instead of modifying an existing entry.'
            );
        });
    }

    public function payrollRun(): BelongsTo
    {
        return $this->belongsTo(PayrollRun::class);
    }

    public function employee(): BelongsTo
    {
        return $this->belongsTo(Employee::class);
    }
}
```

9. Leave

FILE: `app/Domain/Leave/LeaveType.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Leave;

use App\Domain\Tenancy\Company;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\SoftDeletes;

final class LeaveType extends Model
{
    use BelongsToTenant, HasFactory, SoftDeletes;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'company_id', 'name', 'annual_entitlement_days',
        'accrual_rate_per_month', 'requires_approval', 'paid',
    ];

    protected function casts(): array
    {
        return [
            'annual_entitlement_days' => 'decimal:2',
            'accrual_rate_per_month' => 'decimal:3',
            'requires_approval' => 'boolean',
            'paid' => 'boolean',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (LeaveType $t) {
            $t->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function company(): BelongsTo
    {
        return $this->belongsTo(Company::class);
    }
}
```

FILE: `app/Domain/Leave/LeaveBalance.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Leave;

use App\Domain\Employees\Employee;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

final class LeaveBalance extends Model
{
    use BelongsToTenant, HasFactory;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'employee_id', 'leave_type_id', 'year',
        'entitled_days', 'accrued_days', 'used_days',
    ];

    protected function casts(): array
    {
        return [
            'entitled_days' => 'decimal:2',
            'accrued_days' => 'decimal:2',
            'used_days' => 'decimal:2',
            'available_days' => 'decimal:2', // generated column — read-only, never assigned directly
        ];
    }

    protected $guarded_computed = ['available_days'];

    protected static function booted(): void
    {
        static::creating(function (LeaveBalance $b) {
            $b->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function employee(): BelongsTo
    {
        return $this->belongsTo(Employee::class);
    }

    public function leaveType(): BelongsTo
    {
        return $this->belongsTo(LeaveType::class);
    }
}
```

FILE: `app/Domain/Leave/LeaveRequest.php`

```

<?php

declare(strict_types=1);

namespace App\Domain\Leave;

use App\Domain\Employees\Employee;
use App\Domain\Identity\User;
use App\Domain\Leave\Enums\LeaveRequestStatus;
use App\Domain\Leave\Exceptions\InvalidLeaveRequestTransitionException;
use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

final class LeaveRequest extends Model
{
    use BelongsToTenant, HasFactory;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'employee_id', 'leave_type_id', 'start_date', 'end_date',
        'days_requested', 'reason', 'status', 'approved_by', 'approved_at', 'rejection_reason',
    ];

    protected function casts(): array
    {
        return [
            'start_date' => 'date',
            'end_date' => 'date',
            'days_requested' => 'decimal:2',
            'status' => LeaveRequestStatus::class,
            'approved_at' => 'datetime',
        ];
    }

    protected static function booted(): void
    {
        static::creating(function (LeaveRequest $r) {
            $r->id ??= (string) \Illuminate\Support\Str::uuid();
        });
    }

    public function employee(): BelongsTo
    {
        return $this->belongsTo(Employee::class);
    }

    public function leaveType(): BelongsTo
    {
        return $this->belongsTo(LeaveType::class);
    }

    public function approver(): BelongsTo
    {
        return $this->belongsTo(User::class, 'approved_by');
    }

    public function transitionTo(LeaveRequestStatus $next, array $extra = []): void
    {
        if (! in_array($next, $this->status->allowedNextStates(), true)) {
            throw new InvalidLeaveRequestTransitionException($this->status, $next);
        }

        $this->update(array_merge(['status' => $next], $extra));
    }
}

```

FILE: app/Domain/Leave/Exceptions/InvalidLeaveRequestTransitionException.php

```
<?php

declare(strict_types=1);

namespace App\Domain\Leave\Exceptions;

use App\Domain\Leave\Enums\LeaveRequestStatus;

final class InvalidLeaveRequestTransitionException extends \DomainException
{
    public function __construct(public readonly LeaveRequestStatus $from, public readonly LeaveRequestStatus $to)
    {
        parent::__construct("Cannot transition leave request from '{$from->value}' to '{$to->value}'");
    }
}
```

10. Audit

FILE: `app/Domain/Audit/AuditLog.php`

```
<?php

declare(strict_types=1);

namespace App\Domain\Audit;

use App\Domain\Identity\User;
use App\Domain\Tenancy\Tenant;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\MorphTo;

final class AuditLog extends Model
{
    public $incrementing = false;

    protected $keyType = 'string';

    public $timestamps = false; // only created_at exists, handled explicitly below

    protected $fillable = [
        'id', 'tenant_id', 'user_id', 'action', 'auditable_type', 'auditable_id',
        'before', 'after', 'ip_address', 'user_agent', 'created_at',
    ];

    protected function casts(): array
    {
        return ['before' => 'array', 'after' => 'array', 'created_at' => 'datetime'];
    }

    protected static function booted(): void
    {
        static::creating(function (AuditLog $log) {
            $log->id ??= (string) \Illuminate\Support\Str::uuid();
            $log->created_at ??= now();
        });
    }

    public function tenant(): BelongsTo
    {
        return $this->belongsTo(Tenant::class);
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    public function auditable(): MorphTo
    {
        return $this->morphTo();
    }
}
```

FILE: `app/Domain/Audit/Concerns/Auditable.php`

The one system-wide Observer this project uses (per Part A §A.4's stated rule: Observers are rare and deliberate). Redacts any attribute the owning model casts as `encrypted` — those never appear in `before/after`, closing the “sensitive data leaked into an audit trail” risk this task explicitly calls out.


```

<?php

declare(strict_types=1);

namespace App\Domain\Audit\Concerns;

use App\Domain\Audit\AuditLog;
use App\Domain\Tenancy\TenantContext;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Request as RequestFacade;

trait Auditable
{
    protected static function bootAuditable(): void
    {
        static::created(fn (Model $model) => self::recordAudit($model, 'created', null, $model->toArray()));

        static::updated(function (Model $model) {
            $before = array_intersect_key($model->getOriginal(), $model->getChanges());
            $after = $model->getChanges();
            self::recordAudit($model, 'updated', $before, $after);
        });

        static::deleted(fn (Model $model) => self::recordAudit($model, 'deleted', $model->toArray(), null));
    }

    private static function recordAudit(Model $model, string $verb, ?array $before, ?array $after): void
    {
        $encryptedAttributes = method_exists($model, 'casts')
            ? array_keys(array_filter($model->casts(), fn ($cast) => $cast === 'encrypted'))
            : [];

        $redact = fn (?array $data) => $data === null ? null : collect($data)
            ->except($encryptedAttributes)
            ->except(['password', 'remember_token', 'two_factor_secret', 'two_factor_recovery_codes'])
            ->all();

        AuditLog::create([
            'tenant_id' => app(TenantContext::class)->id(),
            'user_id' => Auth::id(),
            'action' => class_basename($model).'.'.$verb,
            'auditable_type' => $model::class,
            'auditable_id' => $model->getKey(),
            'before' => $redact($before),
            'after' => $redact($after),
            'ip_address' => RequestFacade::ip(),
            'user_agent' => RequestFacade::userAgent(),
        ]);
    }
}

```

11. Idempotency

FILE: `app/Support/Idempotency/IdempotencyKey.php`

```
<?php

declare(strict_types=1);

namespace App\Support\Idempotency;

use App\Domain\Tenancy\Concerns\BelongsToTenant;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

final class IdempotencyKey extends Model
{
    use BelongsToTenant, HasFactory;

    public $incrementing = false;

    protected $keyType = 'string';

    protected $fillable = [
        'id', 'tenant_id', 'key', 'route', 'request_hash', 'response_status',
        'response_body', 'status', 'expires_at',
    ];

    protected function casts(): array
    {
        return ['response_body' => 'array', 'expires_at' => 'datetime'];
    }

    protected static function booted(): void
    {
        static::creating(function (IdempotencyKey $k) {
            $k->id ??= (string) \Illuminate\Support\Str::uuid();
            $k->expires_at ??= now()->addHours(24);
        });
    }

    public function isExpired(): bool
    {
        return $this->expires_at->isPast();
    }
}
```

(The reusable `IdempotencyMiddleware` that reads/writes this model — handling “same key + same request” replay, “same key + different request” rejection, and the concurrent-request race — is Application-layer infrastructure, not a database concern; it ships in Stage 2 alongside authentication, since both are request-pipeline middleware built and tested together.)

12. Factories

FILE: `database/factories/TenantFactory.php`

```
<?php

declare(strict_types=1);

namespace Database\Factories;

use App\Domain\Tenancy\Tenant;
use Illuminate\Database\Eloquent\Factories\Factory;

/** @extends Factory<Tenant> */
final class TenantFactory extends Factory
{
    protected $model = Tenant::class;

    public function definition(): array
    {
        $name = fake()->unique()->company();

        return [
            'id' => (string) \Illuminate\Support\Str::uuid(),
            'name' => $name,
            'slug' => \Illuminate\Support\Str::slug($name).'-' . fake()->unique()->numberBetween(1000, 9999),
            'branding' => ['logo_url' => null, 'accent_color' => '#1e3a8a'],
            'status' => 'active',
        ];
    }
}
```

FILE: database/factories/CompanyFactory.php

```
<?php

declare(strict_types=1);

namespace Database\Factories;

use App\Domain\Tenancy\Company;
use App\Domain\Tenancy\Tenant;
use Illuminate\Database\Eloquent\Factories\Factory;

/** @extends Factory<Company> */
final class CompanyFactory extends Factory
{
    protected $model = Company::class;

    public function definition(): array
    {
        $country = fake()->randomElement(['KE', 'NG', 'ZA']);
        $currency = match ($country) {
            'KE' => 'KES', 'NG' => 'NGN', 'ZA' => 'ZAR',
        };

        return [
            'id' => (string) \Illuminate\Support\Str::uuid(),
            'tenant_id' => Tenant::factory(),
            'name' => fake()->company(),
            'country' => $country,
            'currency' => $currency,
            'registration_number' => fake()->numerify('REG-#####'),
        ];
    }

    public function kenya(): self
    {
        return $this->state(['country' => 'KE', 'currency' => 'KES']);
    }
}
```

FILE: database/factories/UserFactory.php

```
<?php

declare(strict_types=1);

namespace Database\Factories;

use App\Domain\Identity\User;
use App\Domain\Tenancy\Tenant;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Facades\Hash;

/** @extends Factory<User> */
final class UserFactory extends Factory
{
    protected $model = User::class;

    public function definition(): array
    {
        return [
            'id' => (string) \Illuminate\Support\Str::uuid(),
            'tenant_id' => Tenant::factory(),
            'first_name' => fake()->firstName(),
            'last_name' => fake()->lastName(),
            'email' => fake()->unique()->safeEmail(),
            'email_verified_at' => now(),
            'password' => Hash::make('password'),
        ];
    }

    public function unverified(): self
    {
        return $this->state(['email_verified_at' => null]);
    }
}
```

FILE: database/factories/EmployeeFactory.php

```
<?php

declare(strict_types=1);

namespace Database\Factories;

use App\Domain\Employees\Employee;
use App\Domain\Tenancy\Company;
use Illuminate\Database\Eloquent\Factories\Factory;

/** @extends Factory<Employee> */
final class EmployeeFactory extends Factory
{
    protected $model = Employee::class;

    public function definition(): array
    {
        $company = Company::factory();

        return [
            'id' => (string) \Illuminate\Support\Str::uuid(),
            'company_id' => $company,
            'employee_number' => 'EMP-' . fake()->unique()->numerify('#####'),
            'first_name' => fake()->firstName(),
            'last_name' => fake()->lastName(),
            'email' => fake()->unique()->safeEmail(),
            'country' => 'KE',
            'hire_date' => fake()->dateTimeBetween('-3 years', 'now'),
            'status' => 'active',
        ];
    }

    public function terminated(): self
    {
        return $this->state(fn () => [
            'status' => 'terminated',
            'termination_date' => fake()->dateTimeBetween('-6 months', 'now'),
        ]);
    }
}
```

FILE: `database/factories/SalaryStructureFactory.php`

```
<?php

declare(strict_types=1);

namespace Database\Factories;

use App\Domain\Payroll\SalaryStructure;
use App\Domain\Tenancy\Company;
use Illuminate\Database\Eloquent\Factories\Factory;

/** @extends Factory<SalaryStructure> */
final class SalaryStructureFactory extends Factory
{
    protected $model = SalaryStructure::class;

    public function definition(): array
    {
        return [
            'id' => (string) \Illuminate\Support\Str::uuid(),
            'company_id' => Company::factory(),
            'name' => fake()->randomElement(['Junior Grade', 'Mid Grade', 'Senior Grade', 'Executive']),
            'basic_salary' => fake()->numberBetween(40, 400) * 1000,
            'currency' => 'KES',
        ];
    }
}
```

FILE: `database/factories/LeaveTypeFactory.php`

```
<?php

declare(strict_types=1);

namespace Database\Factories;

use App\Domain\Leave\LeaveType;
use App\Domain\Tenancy\Company;
use Illuminate\Database\Eloquent\Factories\Factory;

/** @extends Factory<LeaveType> */
final class LeaveTypeFactory extends Factory
{
    protected $model = LeaveType::class;

    public function definition(): array
    {
        return [
            'id' => (string) \Illuminate\Support\Str::uuid(),
            'company_id' => Company::factory(),
            'name' => fake()->randomElement(['Annual Leave', 'Sick Leave', 'Maternity Leave', 'Compassionate Leave']),
            'annual_entitlement_days' => 21,
            'accrual_rate_per_month' => 1.75,
            'requires_approval' => true,
            'paid' => true,
        ];
    }
}
```

13. Seeders

FILE: `database/seeders/RolePermissionSeeder.php`

Defines the permission catalog this whole project's Policies check against (Stage 2 wires the Gates/Policies themselves — this seeder is the source of truth for *which permission strings exist*). Run once per tenant at tenant-creation time in production (Stage 2's `CreateTenantCommand`); run globally here for local/dev/test.

```

<?php

declare(strict_types=1);

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Spatie\Permission\Models\Permission;
use Spatie\Permission\Models\Role;
use Spatie\Permission\PermissionRegistrar;

final class RolePermissionSeeder extends Seeder
{
    /**
     * The permission catalog for Stage 1's domains. Extended in later stages as
     * Loans/Attendance/Documents/Billing/etc. are implemented — this list is not
     * final, it is exactly what Stage 1-3's endpoints need and nothing invented ahead of use.
     */
    private const PERMISSIONS = [
        'employees:read', 'employees:write',
        'payroll:read', 'payroll:run', 'payroll:approve',
        'leave:read', 'leave:write', 'leave:approve',
        'compliance:read',
        'audit:read',
    ];

    private const ROLE_PERMISSIONS = [
        'admin' => self::PERMISSIONS, // every permission above
        'hr' => [
            'employees:read', 'employees:write',
            'payroll:read', 'payroll:run',
            'leave:read', 'leave:write', 'leave:approve',
            'compliance:read',
        ],
        'employee' => [
            // Deliberately empty: an employee's access to their OWN records is granted by
            // ownership checks in Policies (Stage 2/3), not by a blanket permission — see
            // Part A §A.4's "ownership vs role" distinction, applied literally here.
        ],
    ];

    public function run(): void
    {
        app(PermissionRegistrar::class)->forgetCachedPermissions();

        foreach (self::PERMISSIONS as $permission) {
            Permission::firstOrCreate(['name' => $permission, 'guard_name' => 'sanctum']);
        }

        foreach (self::ROLE_PERMISSIONS as $roleName => $permissions) {
            $role = Role::firstOrCreate(['name' => $roleName, 'guard_name' => 'sanctum']);
            $role->syncPermissions($permissions);
        }
    }
}

```


FILE: database/seeder/DemoTenantSeeder.php

```

<?php

declare(strict_types=1);

namespace Database\Seeders;

use App\Domain\Employees\Employee;
use App\Domain\Identity\User;
use App\Domain\Leave\LeaveBalance;
use App\Domain\Leave\LeaveType;
use App\Domain\Payroll\SalaryStructure;
use App\Domain\Tenancy\Company;
use App\Domain\Tenancy\Tenant;
use App\Domain\Tenancy\TenantContext;
use Illuminate\Database\Seeder;

final class DemoTenantSeeder extends Seeder
{
    public function run(): void
    {
        $tenant = Tenant::create([
            'name' => 'Acme Africa Ltd',
            'slug' => 'acme-africa',
            'status' => 'active',
        ]);

        // Seeding writes tenant-scoped rows, so tenant context must be bound exactly
        // the way request middleware binds it in production (Part 8 §8.4 of the
        // architecture doc — this is not a special seeder-only shortcut).
        app(TenantContext::class)->set($tenant->id);

        $company = Company::create([
            'tenant_id' => $tenant->id,
            'name' => 'Acme Kenya',
            'country' => 'KE',
            'currency' => 'KES',
        ]);

        $admin = User::create([
            'tenant_id' => $tenant->id,
            'first_name' => 'Amara',
            'last_name' => 'Otieno',
            'email' => 'admin@acme.test',
            'email_verified_at' => now(),
            'password' => \Illuminate\Support\Facades\Hash::make('password'),
        ]);
        $admin->assignRole('admin');

        $hr = User::create([
            'tenant_id' => $tenant->id,
            'first_name' => 'Brian',
            'last_name' => 'Mwangi',
            'email' => 'hr@acme.test',
            'email_verified_at' => now(),
            'password' => \Illuminate\Support\Facades\Hash::make('password'),
        ]);
        $hr->assignRole('hr');

        $structure = SalaryStructure::create([
            'company_id' => $company->id,
            'name' => 'Mid Grade',
            'basic_salary' => 120000,
            'currency' => 'KES',
        ]);

        $manager = Employee::create([
            'company_id' => $company->id,
            'employee_number' => 'EMP-0001',
            'first_name' => 'Chidi',
            'last_name' => 'Nwosu',
            'email' => 'chidi@acme.test',
        ]);
    }
}

```

```

        'country' => 'KE',
        'hire_date' => now()->subYears(2),
        'status' => 'active',
        'salary_structure_id' => $structure->id,
    ));

    Employee::create([
        'company_id' => $company->id,
        'employee_number' => 'EMP-0002',
        'first_name' => 'Wanjiru',
        'last_name' => 'Kamau',
        'email' => 'wanjiru@acme.test',
        'country' => 'KE',
        'hire_date' => now()->subMonths(8),
        'status' => 'active',
        'salary_structure_id' => $structure->id,
        'manager_id' => $manager->id,
    ]);

    $annualLeave = LeaveType::create([
        'company_id' => $company->id,
        'name' => 'Annual Leave',
        'annual_entitlement_days' => 21,
        'accrual_rate_per_month' => 1.75,
        'requires_approval' => true,
        'paid' => true,
    ]);

    foreach (Employee::query()->get() as $employee) {
        LeaveBalance::create([
            'employee_id' => $employee->id,
            'leave_type_id' => $annualLeave->id,
            'year' => now()->year,
            'entitled_days' => 21,
            'accrued_days' => 14,
            'used_days' => 0,
        ]);
    }

    app(TenantContext::class)->clear();

    $this->command->info("Demo tenant seeded. Login: admin@acme.test / password");
}
}

```

FILE: `database/seeder/DatabaseSeeder.php`

```

<?php

declare(strict_types=1);

namespace Database\Seeders;

use Illuminate\Database\Seeder;

final class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call([
            RolePermissionSeeder::class,
            DemoTenantSeeder::class,
        ]);
    }
}

```

14. Stage 1 Completeness Check



Every model above has a migration (verified 1:1 — 15 tables + RBAC's 5 tables + Sanctum's 1 table)

□
Every foreign key references a migration that runs earlier in the numbered sequence (verified: `companies.tenant_id` → 0001; `employees.company_id` → 0002; `employees.salary_structure_id` → 0008; `payroll_runs.payroll_period_id` → 0012; `payroll_entries.payroll_run_id` → 0013; `leave_balances.leave_type_id` → 0015, etc.)

□
Required composite indexes exist per this task's explicit list: `tenant_id` alone (every table), `tenant_id+status` (`payroll_runs`, `payroll_periods`), `tenant_id+employee_id` (`payroll_entries`, `leave_balances`), `tenant_id+created_at` (`audit_logs`), `idempotency_key` (`idempotency_keys.tenant_id+key+route`)

□
RLS enabled and forced on every tenant-scoped table, with policies verified to reference the same `app.current_tenant_id` session variable. `TenantContext` sets

□
Payroll immutability implemented at the schema level (no `payroll_entries.updated_at`) **and** the model level (`UPDATE_AT = null` + a hard exception on updating)

□
Money columns are `decimal(14,2)` / `decimal(16,2)`, never `float` / `double`, anywhere in Stage 1

□
Every [NEW — confirm] table from Part A that's in scope for Stage 1 (`payroll_periods`) is explicitly labeled in both the schema table and its migration file — nothing invented silently

Stage 1 Persistent Implementation Checklist

COMPLETED (this response):

Migrations: 0001-tenants, 0002-companies, 0003-users, 0004-permission_tables, 0005-personal_access_tokens, 0006-employees, 0007-contracts, 0008-salary_structures, 0009-salary_components, 0010-salary_structure_components, 0011-onboarding_tasks, 0012-payroll_periods, 0013-payroll_runs, 0014-payroll_entries, 0015-leave_types, 0016-leave_balances, 0017-leave_requests, 0018-audit_logs, 0019-idempotency_keys, 0020-enable_row_level_security

Enums: EmployeeStatus, ContractType, SalaryComponentType, SalaryComponentCalcType, PayrollRunStatus, PayrollPeriodStatus, LeaveRequestStatus

Foundation: TenantContext, TenantScope, BelongsToTenant trait

Models: Tenant, Company, User, Employee, Contract, OnboardingTask, SalaryStructure, SalaryComponent, SalaryStructureComponent (pivot), Money (value object), PayrollPeriod, PayrollRun (+ transitionTo), PayrollEntry (immutable), LeaveType, LeaveBalance, LeaveRequest (+ transitionTo), AuditLog, Auditable trait, IdempotencyKey

Factories: Tenant, Company, User, Employee, SalaryStructure, LeaveType

Seeders: RolePermissionSeeder, DemoTenantSeeder, DatabaseSeeder

REMAINING (Stage 1 continuation – next response):

Migrations: loan_products, loans, loan_repayments, attendance_records, public_holidays, employee_documents, plans, subscriptions, invoices, usage_records, payment_transactions, payment_provider_events, notifications, notification_preferences, push_subscriptions, webhook_endpoints, webhook_delivery_logs, accounting_connections, accounting_sync_jobs, invitations

Models + factories for all of the above

ERD (Mermaid + ASCII) covering the FULL schema (Stage 1 + continuation)

THEN:

Stage 2 – Backend foundation (auth, tenancy middleware, RBAC middleware, IdempotencyMiddleware, API error/response conventions, OpenAPI foundation)

Stage 3 – Domain-by-domain APIs (Employees → Payroll engine → ... → all 15 domains)

Stage 4 – OpenAPI + generated TypeScript

Stage 5 – Frontend integration

Stage 6 – Testing

Stage 7 – Docker/Nginx/Observability/CI

CONTINUE FROM:
database/migrations/2025_01_01_000021_create_loan_products_table.php

COMPLETED:

Stage 1 – Tenancy, Identity, RBAC, Employees, Compensation, Payroll (periods/runs/entries), Leave, Audit, Idempotency: 20 migrations, 7 enums, tenancy foundation, 18 models, 6 factories, 3 seeders – all real, complete, runnable files.

REMAINING:

Stage 1 continuation – Loans, Attendance, Documents, Billing, Payments, Notifications, Webhooks, Accounting, Invitations (migrations + models + factories), then the full ERD. After that: Stage 2 (backend foundation) through Stage 7 (infrastructure), per the required generation order.